Experiment 1: Install Matplotlib and Display a Simple Line Graph

Python Program

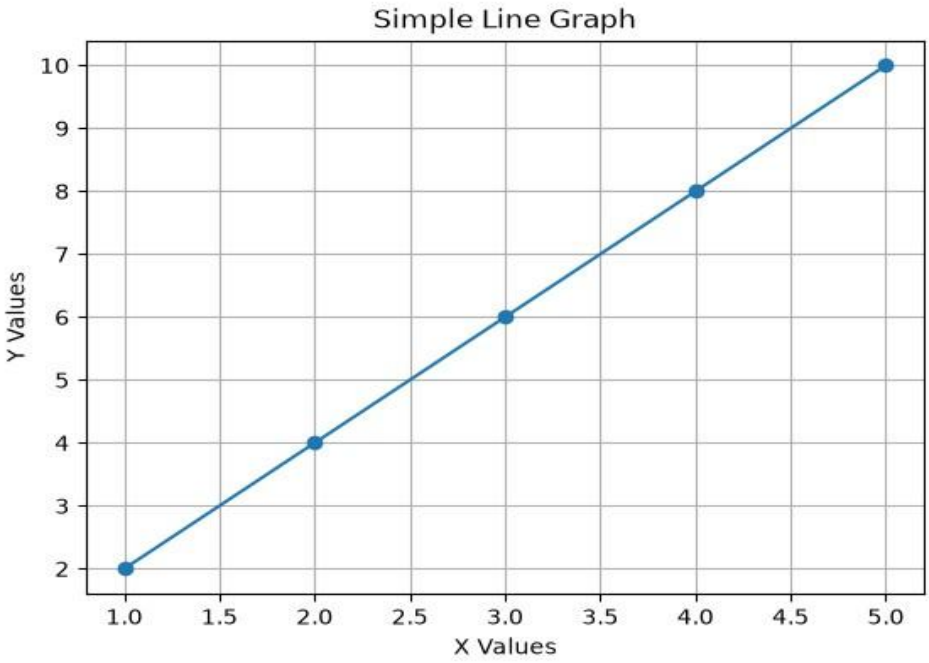
```
import matplotlib.pyplot as plt


x = [1, 2, 3, 4, 5] y
= [2, 4, 6, 8, 10]


plt.plot(x, y, marker="o")
plt.title("Simple Line Graph")
plt.xlabel("X Values") plt.ylabel("Y
Values") plt.grid(True)

plt.show()
```

Output



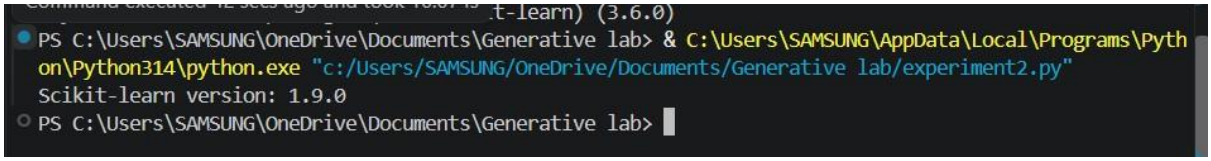
Experiment 2: Install Scikit-learn and Print Its Version

Python Program

```
import sklearn

print("Scikit-learn version:", sklearn.__version__)
```

Output



Experiment 3: Create a NumPy Array Containing the First 20 Natural Numbers

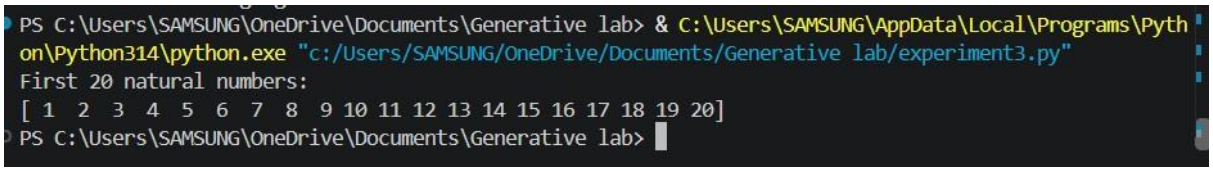
Python Program

```
import numpy as np

arr = np.arange(1, 21) print("First

20 natural numbers:") print(arr)
```

Output



Experiment 4: Create a Pandas DataFrame with Student Names and Grades

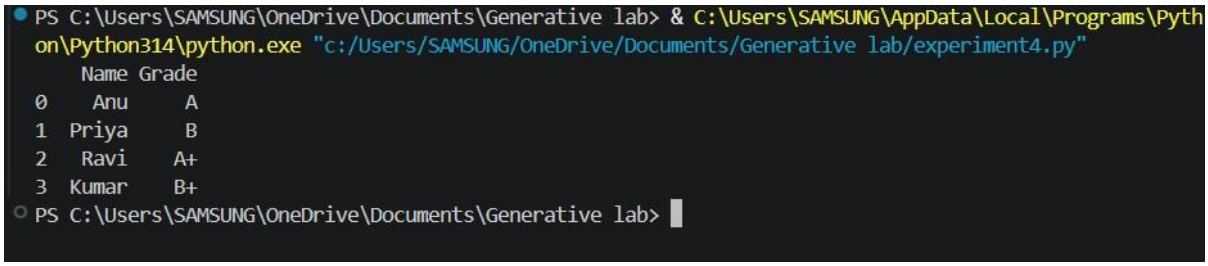
Python Program

```
import pandas as pd
```

```
data = {
   "Name": ["Anu", "Priya", "Ravi", "Kumar"],
   "Grade": ["A", "B", "A+", "B+"]
}
```

df = pd.DataFrame(data) print(df)

Output



Experiment 5: Verify the Installed Versions of All AI Libraries on Your System

Python Program

```
import numpy as np
import pandas as pd
import matplotlib import
sklearn

print("NumPy:", np.__version__) print("Pandas:",
pd.__version__) print("Matplotlib:",
matplotlib.__version__) print("Scikit-learn:",
sklearn.__version__)
```

Output



Experiment 6: Create Tensors Using PyTorch and Perform Basic Mathematical Operations

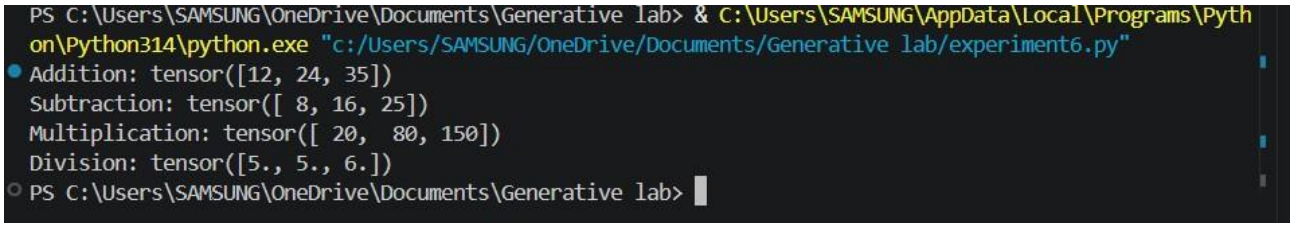
Python Program

```
import torch


a = torch.tensor([10, 20, 30]) b
= torch.tensor([2, 4, 5])


print("Addition:", a + b) print("Subtraction:",
a - b) print("Multiplication:", a * b)
print("Division:", a / b)
```

Output



Experiment 7: Load a Pre-trained Transformer Model Using Hugging Face and Perform Sentiment Analysis

Python Program

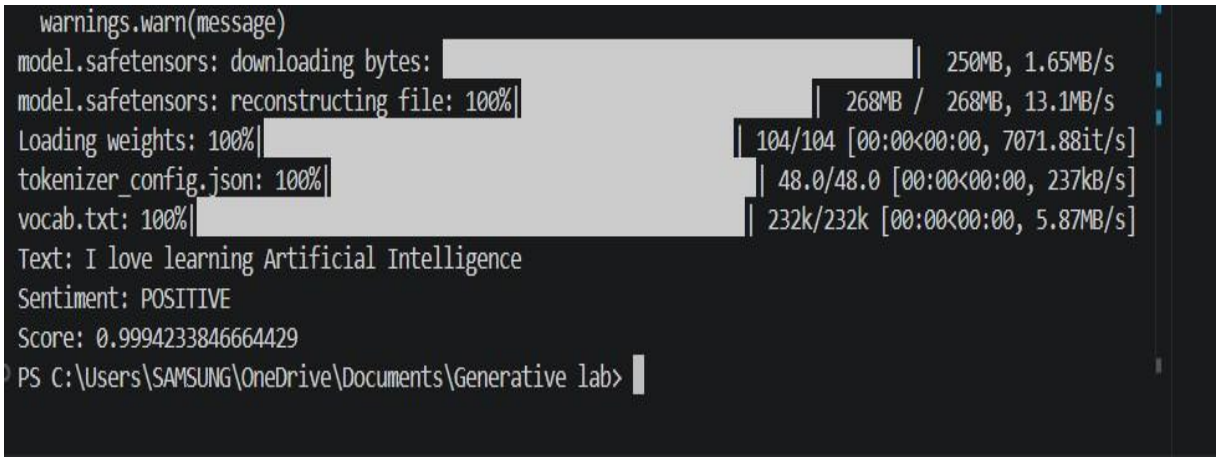
```
from transformers import pipeline


sentiment = pipeline("sentiment-analysis")


text = "I love learning Artificial Intelligence" result
= sentiment(text)


print("Text:", text)
print("Sentiment:", result[0]["label"]) print("Score:",
result[0]["score"])
```

Output



Experiment 8: Tokenization Using BERT Tokenizer

Python Program

```
from transformers import BertTokenizer

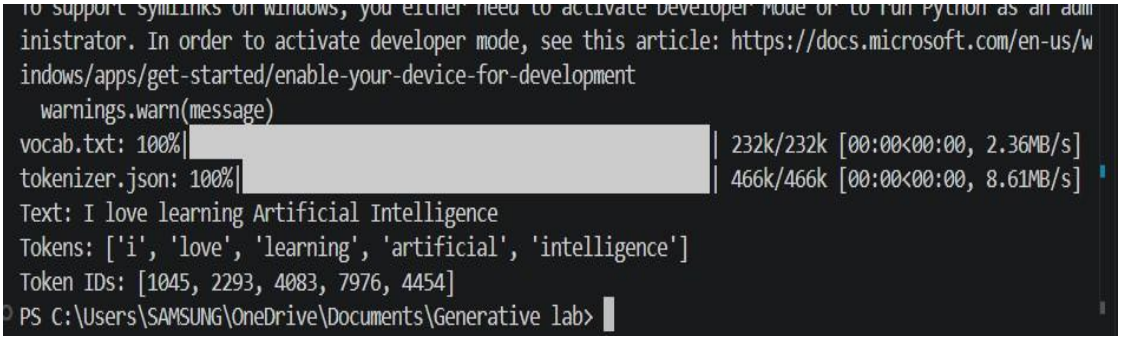
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

text = "I love learning Artificial Intelligence"

tokens = tokenizer.tokenize(text) ids =
tokenizer.convert_tokens_to_ids(tokens)

print("Text:",          text)
print("Tokens:", tokens)
print("Token IDs:", ids)
```

Output



Experiment 9: Generate Text Using a Pre-trained GPT-2 Model

Python Program

```
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

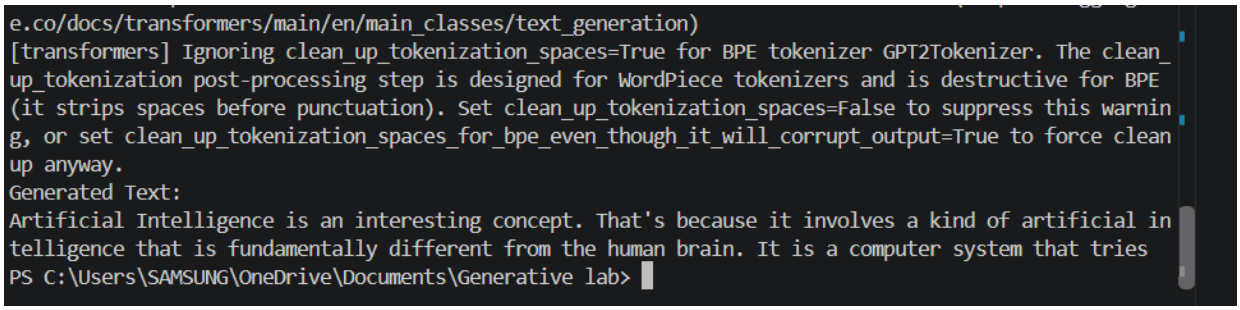
prompt = "Artificial Intelligence is"

result = generator(prompt, max_new_tokens=30, num_return_sequences=1)

print("Generated Text:")

print(result[0]["generated_text"])
```

Output



Experiment 10: Sentiment Analysis Using Hugging Face Pipeline API

Python Program

```
from transformers import pipeline

model = pipeline("sentiment-analysis")

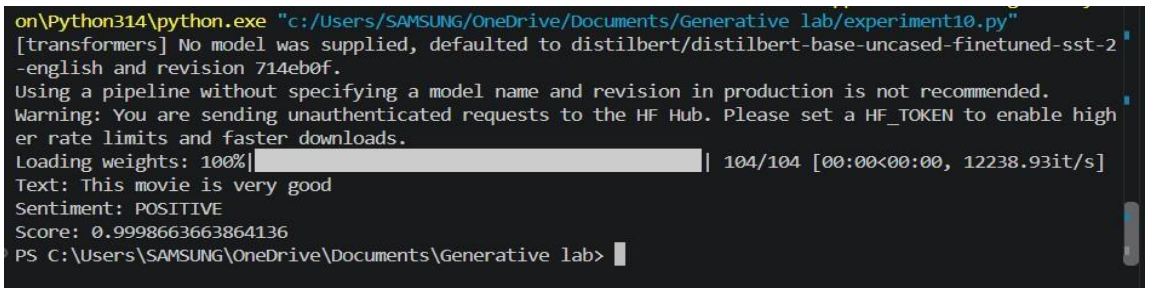
text = "This movie is very good" result

= model(text) print("Text:", text)

print("Sentiment:", result[0]["label"]) print("Score:",

result[0]["score"])
```

Output



Experiment 11: Load the BERT Tokenizer and Tokenize a Sentence

Python Program

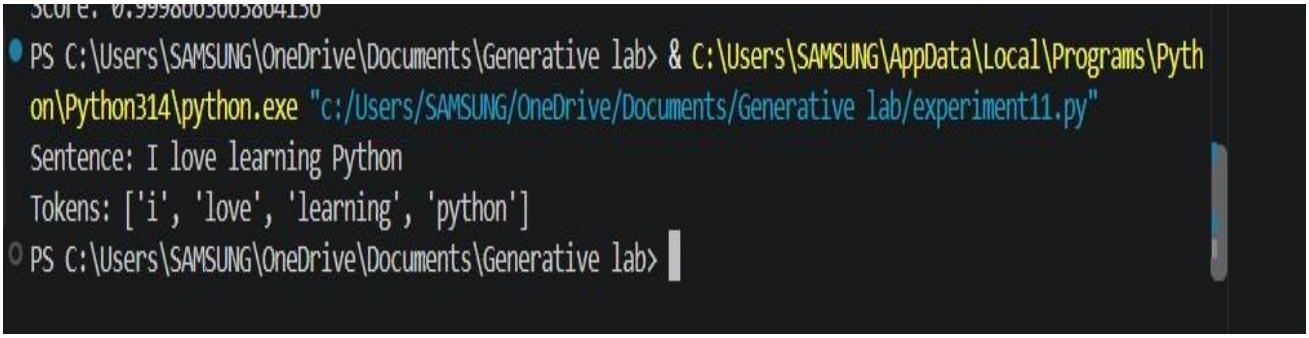
```
from transformers import BertTokenizer tokenizer =

BertTokenizer.from_pretrained("bert-base-uncased") text = "I

love learning Python" tokens = tokenizer.tokenize(text)

print("Sentence:", text) print("Tokens:", tokens)
```

Output



Experiment 12: Load a Pre-trained BERT Model and Generate Contextual Embeddings

Python Program

```
import torch from transformers import

BertTokenizer, BertModel

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

model = BertModel.from_pretrained("bert-base-uncased")

text = "I love learning Python" inputs =

tokenizer(text, return_tensors="pt")

with torch.no_grad():

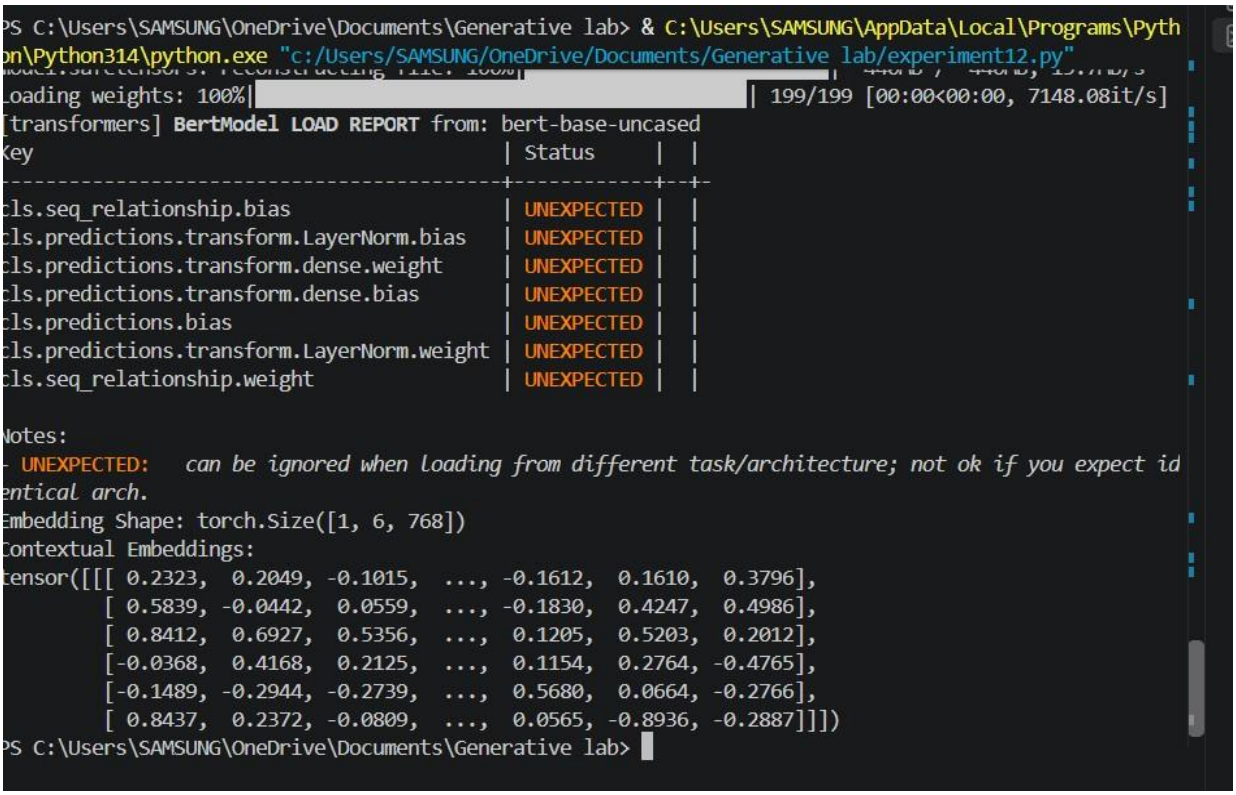
    outputs = model(**inputs)

print("Embedding Shape:", outputs.last_hidden_state.shape)

print("Contextual Embeddings:")

print(outputs.last_hidden_state)
```

output:


PS C:\Users\SAMSUNG\OneDrive\Documents\Generative lab> & C:\Users\SAMSUNG\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/SAMSUNG/OneDrive/Documents/Generative lab/experiment12.py"
Loading weights: 100%| 199/199 [00:00<00:00, 7148.08it/s]
[transformers] BertModel LOAD REPORT from: bert-base-uncased
Key | Status | |
cls.seq_relationship.bias | UNEXPECTED | |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED | |
cls.predictions.transform.dense.weight | UNEXPECTED | |
cls.predictions.transform.dense.bias | UNEXPECTED | |
cls.predictions.bias | UNEXPECTED | |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED | |
cls.seq_relationship.weight | UNEXPECTED | |
Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Embedding Shape: torch.Size([1, 6, 768])
Contextual Embeddings:
tensor([[[0.2323, 0.2049, -0.1015, ..., -0.1612, 0.1610, 0.3796],
[0.5839, -0.0442, 0.0559, ..., -0.1830, 0.4247, 0.4986],
[0.8412, 0.6927, 0.5356, ..., 0.1205, 0.5203, 0.2012],
[-0.0368, 0.4168, 0.2125, ..., 0.1154, 0.2764, -0.4765],
[-0.1489, -0.2944, -0.2739, ..., 0.5680, 0.0664, -0.2766],
[0.8437, 0.2372, -0.0809, ..., 0.0565, -0.8936, -0.2887]]])
PS C:\Users\SAMSUNG\OneDrive\Documents\Generative lab>

Experiment 13: Generate Text Using a Pre-trained GPT-2 Model

Python Program

```
from transformers import pipeline generator =

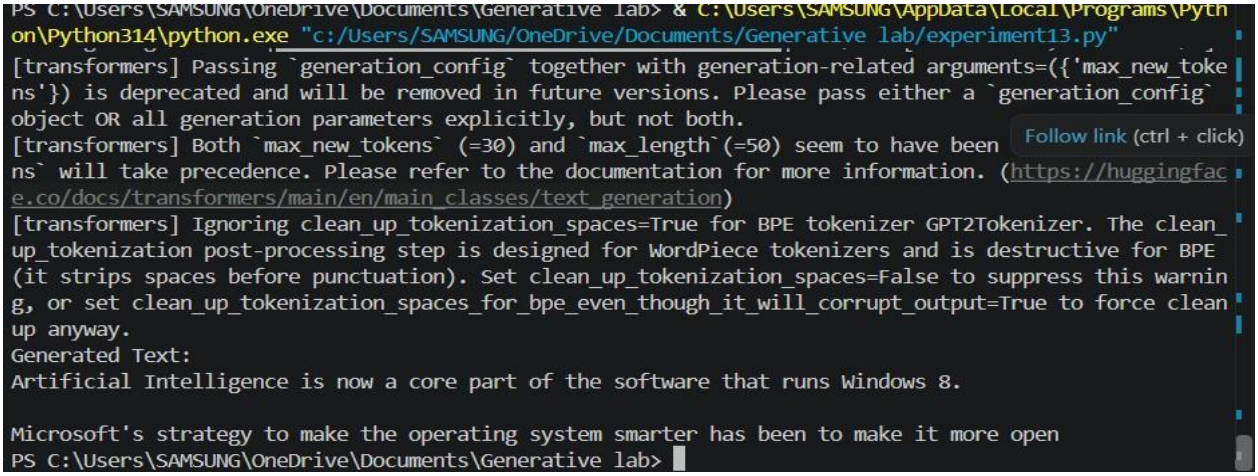
pipeline("text-generation", model="gpt2") text =

"Artificial Intelligence is" result = generator(text,

max_new_tokens=30) print("Generated Text:")

print(result[0]["generated_text"])
```

Output



Experiment 14: Question Answering Using a Pre-trained Transformer

Python Program

```
from transformers import pipeline


qa = pipeline("question-answering")


context = "Python is a popular programming language." question

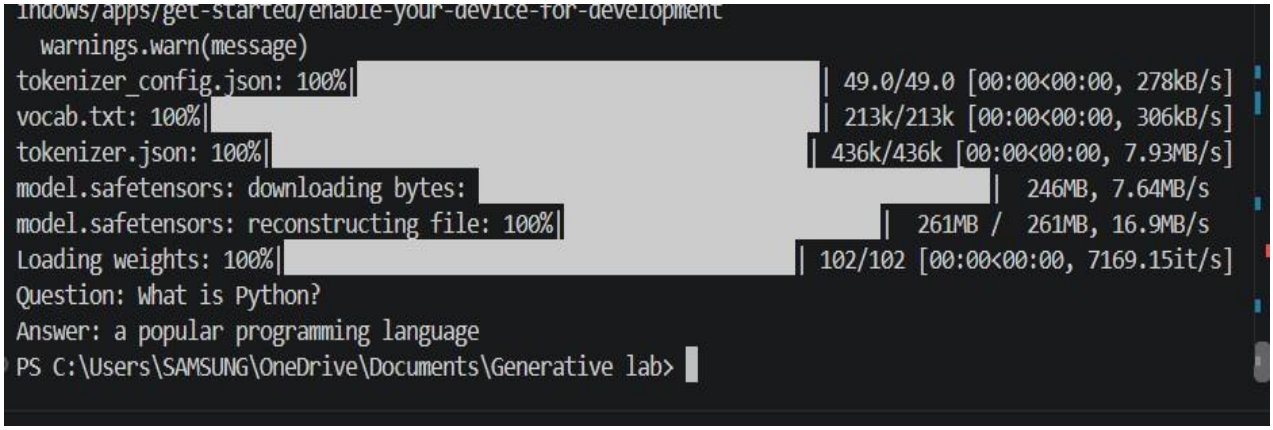
= "What is Python?"


result = qa(question=question, context=context)
```

print("Question:", question)

print("Answer:", result["answer"])

Output



Experiment 15: Tokenize a Sentence Using the BERT Tokenizer and Display Tokens and Token IDs

Python Program

from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-

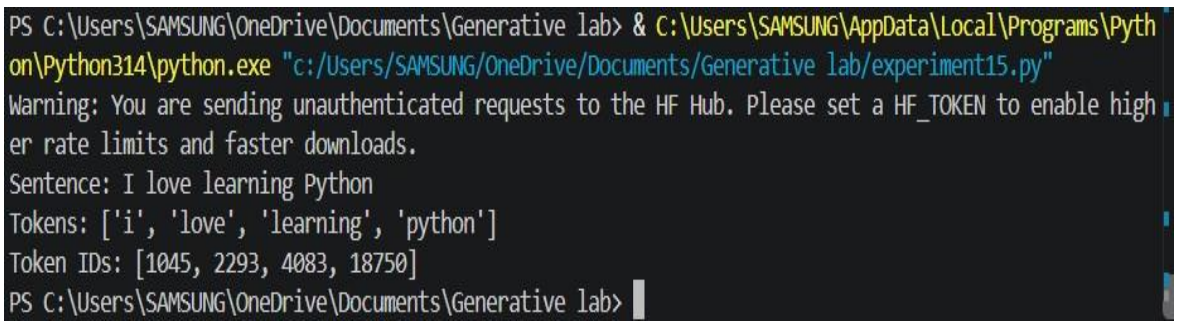
uncased") text = "I love learning Python" tokens =

tokenizer.tokenize(text) ids =

tokenizer.convert_tokens_to_ids(tokens) print("Sentence:",

text) print("Tokens:", tokens) print("Token IDs:", ids)

Output



Experiment 16: Compare the Tokenization Outputs of BERT and GPT-2

Python Program

```
from transformers import BertTokenizer,
GPT2Tokenizer bert =
BertTokenizer.from_pretrained("bert-base-uncased")
gpt2 = GPT2Tokenizer.from_pretrained("gpt2") text = "I
love learning Python" print("BERT Tokens:",
bert.tokenize(text))
print("BERT IDs:", bert.encode(text,
add_special_tokens=False)) print("GPT-2 Tokens:",
gpt2.tokenize(text)) print("GPT-2 IDs:", gpt2.encode(text))
```

Output

```
PS C:\Users\SAMSUNG\OneDrive\Documents\Generative lab> & C:\Users\SAMSUNG\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/SAMSUNG/OneDrive/Documents/Generative lab/experiment16.py"
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
BERT Tokens: ['i', 'love', 'learning', 'python']
BERT IDs: [1045, 2293, 4083, 18750]
GPT-2 Tokens: ['I', 'Ġlove', 'Ġlearning', 'ĠPython']
GPT-2 IDs: [40, 1842, 4673, 11361]
PS C:\Users\SAMSUNG\OneDrive\Documents\Generative lab>
```

Experiment 17: Generate Contextual Embeddings for a Sentence Using BERT

Python Program

```
import torch from transformers import BertTokenizer, BertModel
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased") model =
BertModel.from_pretrained("bert-base-uncased")
```

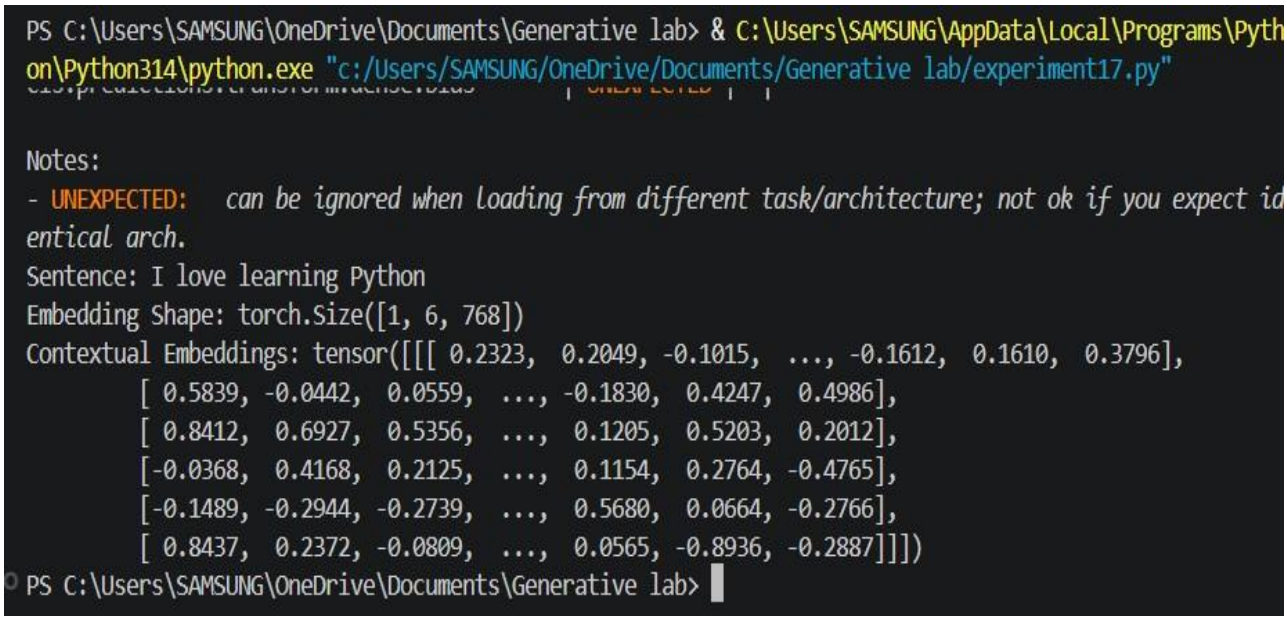
```
text = "I love learning Python" inputs = tokenizer(text, return_tensors="pt")

with torch.no_grad():
    outputs = model(**inputs)

embeddings = outputs.last_hidden_state

print("Sentence:", text) print("Embedding
Shape:", embeddings.shape)
print("Contextual Embeddings:", embeddings)
```

Output



Experiment 18: Compare Contextual Embeddings for Two Semantically Similar Sentences

Python Program

```
import torch from transformers import BertTokenizer, BertModel from torch.nn.functional import
cosine_similarity
```

```
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

model = BertModel.from_pretrained("bert-base-uncased")


s1 = "I love learning Python"

s2 = "I enjoy studying

Python"


def get_embedding(sentence):

    inputs = tokenizer(sentence, return_tensors="pt")

with torch.no_grad():

        output = model(**inputs)

return output.last_hidden_state[:, 0, :]


e1 = get_embedding(s1)

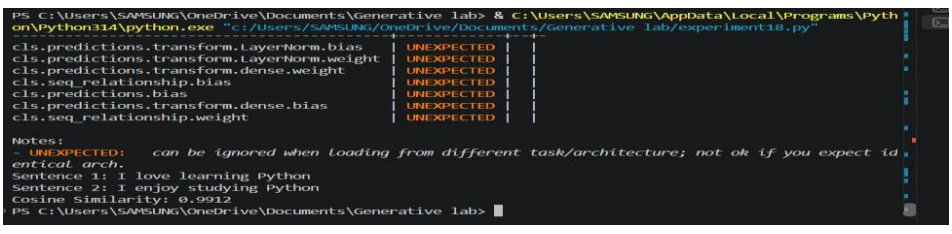
e2 = get_embedding(s2)


score = cosine_similarity(e1, e2).item()


print("Sentence 1:", s1)
print("Sentence 2:", s2) print("Cosine

Similarity:", round(score, 4))
```

Output

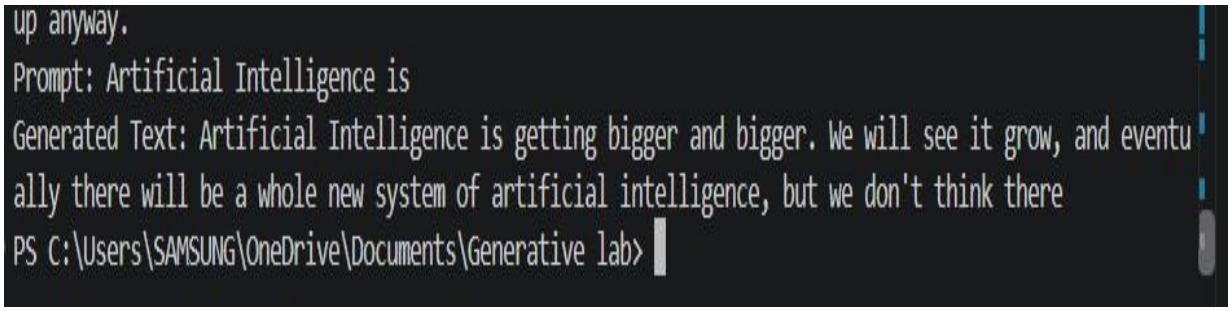


Experiment 19: Generate Text from a Prompt Using GPT-2

Python Program

```
from transformers import pipeline generator = pipeline("text-generation",
model="gpt2") prompt = "Artificial Intelligence is" result =
generator(prompt, max_new_tokens=30, num_return_sequences=1)
print("Prompt:", prompt)
print("Generated Text:", result[0]["generated_text"])
```

Output



Experiment 20: Generate and Compare Responses for Multiple Prompts Using a Pre-trained LLM

Python Program

```
from transformers import pipeline llm =
pipeline("text-generation", model="gpt2")
prompts = [
    "Artificial Intelligence is",
    "Python programming is",
    "Climate change causes"
```

]

for prompt in prompts:

result = llm(prompt, max_new_tokens=20,

do_sample=False, pad_token_id=50256)

print("\nPrompt:", prompt)

print("Response:", result[0]["generated_text"])Output

Output

